

Οδηγός Εργασίας

Python Scrabble Project

ΕΡΓΑΣΙΑ Python

ΘΕΩΡΙΕΣ ΜΑΘΗΣΗΣ & ΕΚΠΑΙΔΕΥΤΙΚΟ ΛΟΓΙΣΜΙΚΟ 2026



Ελλάδα / Το επιτραπέζιο παιχνίδι Scrabble μπαίνει στα σχολεία ως εργαλείο εκμάθησης των ελληνικών

Ο πρόεδρος του Κέντρου Ελληνικής Γλώσσας μιλά για τις ενέργειες του επιστημονικού φορέα του υπουργείου Παιδείας να εντάξει το δημοφιλές επιτραπέζιο παιχνίδι Scrabble στο σχολικό περιβάλλον

NEWSROOM, 4.8.2019 | 11:08



[Σκραμπλ στη Βικιπαίδεια](#)

Περιεχόμενα

- (1) Γενική περιγραφή
- (2) Αρχιτεκτονική λογισμικού
- (3) Κλάσεις
- (4) Κανόνες του παιχνιδιού
- (5) Θέματα ανάπτυξης κώδικα
 - 5.1 Δομή δεδομένων για τις λέξεις
 - 5.2 Αλγόριθμος πολιτικής παιχνιδιού
 - 5.3 Βιβλιοθήκη & import
 - 5.4 Άλλα θέματα ανάπτυξης κώδικα
- (6) Αξιολόγηση εργασίας
 - 6.1 Τεκμηρίωση & docstring
 - 6.2 Κριτήρια Αξιολόγησης
- (7) Παραδοτέο

(1) Γενική περιγραφή



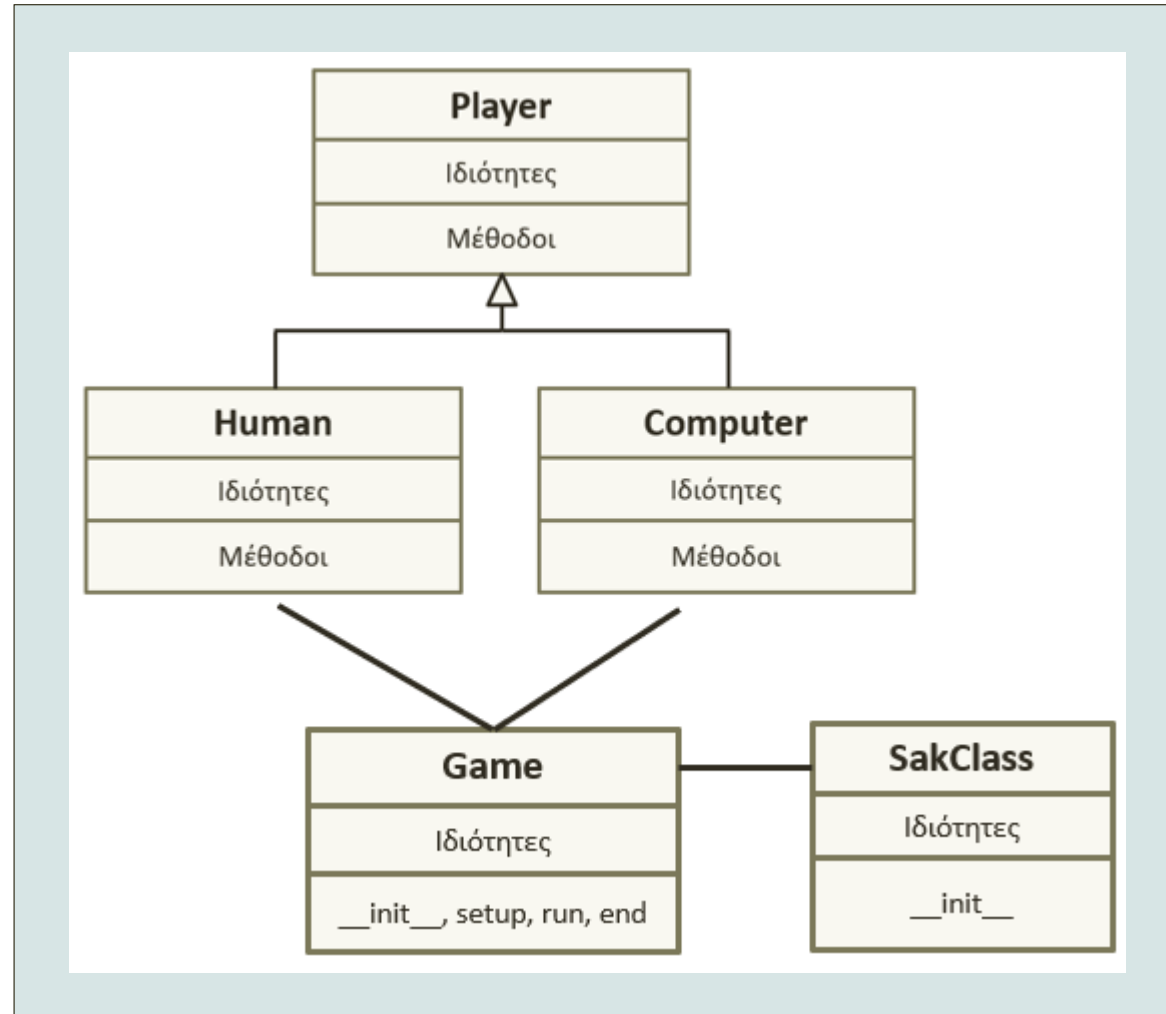
Γενική περιγραφή της εργασίας

- Ο γενικός στόχος της εργασίας είναι να αναπτύξετε μια εφαρμογή σε **αντικειμενοστρεφή κώδικα Python** η οποία να υλοποιεί μια **απλοποιημένη έκδοχή** του παιχνιδιού **Scrabble** στον υπολογιστή.
- Ειδικότερος στόχος της εργασίας είναι να δημιουργηθεί μια βασική **αρχιτεκτονική** και **αλγόριθμοι** που να καθοδηγούν το λογισμικό (τον παίκτη Η/Υ ή και τον παίκτη-άνθρωπο σε κάποιες περιπτώσεις) να παίξει το παιχνίδι
- Η εφαρμογή θα εκτελείται σε «character mode» (δηλ. απλή διεπαφή χαρακτήρων στη γραμμή εντολών) και δεν θα περιλαμβάνει κανενός είδους γραφική διεπαφή (όχι GUI).
- Η εφαρμογή σας θα πρέπει να ακολουθεί σε γενικές γραμμές την ανάλυση που περιγράφεται στις επόμενες διαφάνειες.
- Η εργασία είναι **ατομική**. Κάθε φοιτητής/φοιτήτρια πρέπει να αναπτύξει και να υποβάλει τη δική του/της εργασία.
- *Η εργασία μπορεί να υποβληθεί σε όποια από τις εξής εξεταστικές θέλετε: Ιούνιος ή Σεπτέμβριος 2026 - και για τους/τις επί πτυχίω και Φεβρουάριος 2027)*

(2) Αρχιτεκτονική

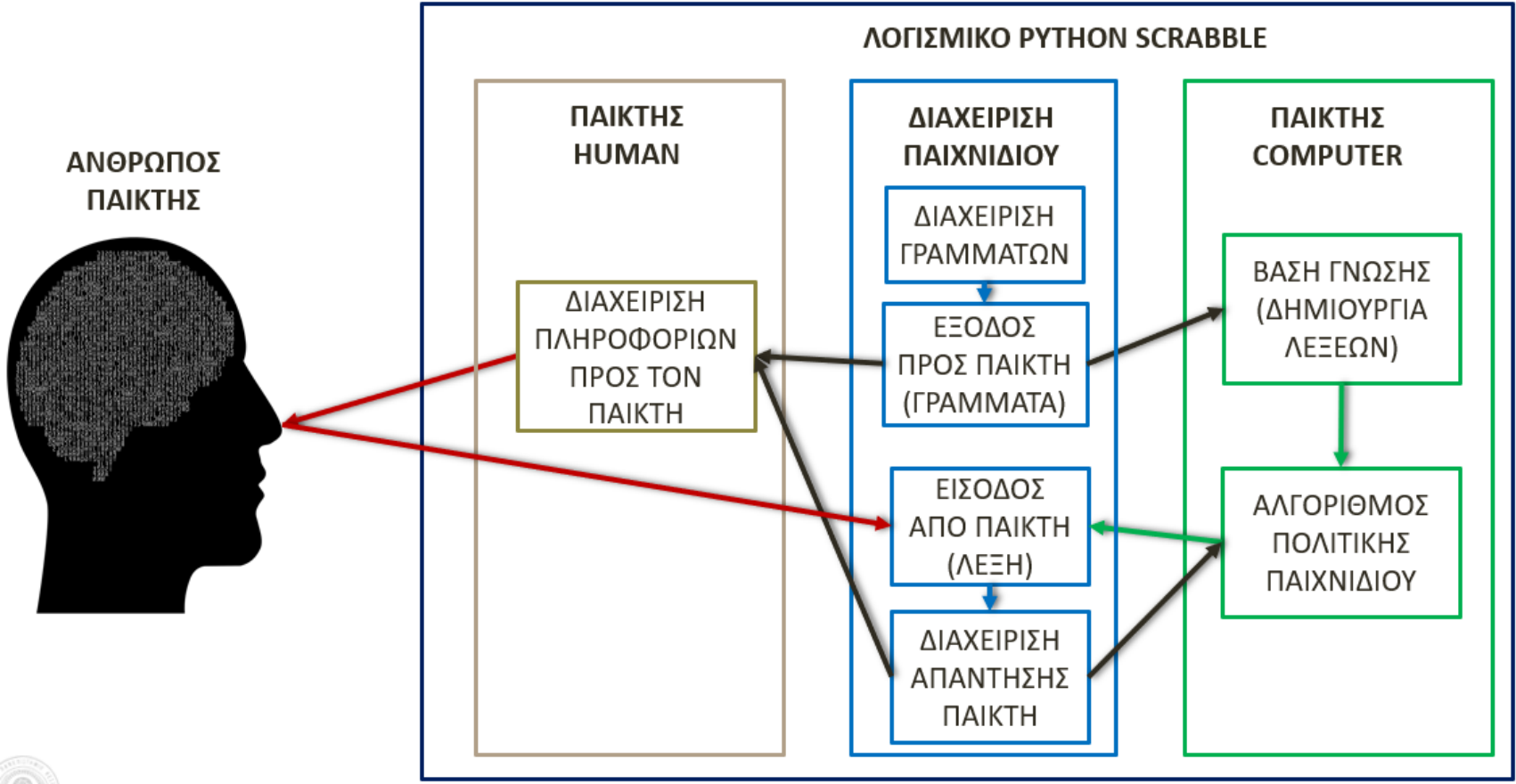


Αρχιτεκτονική 1/2



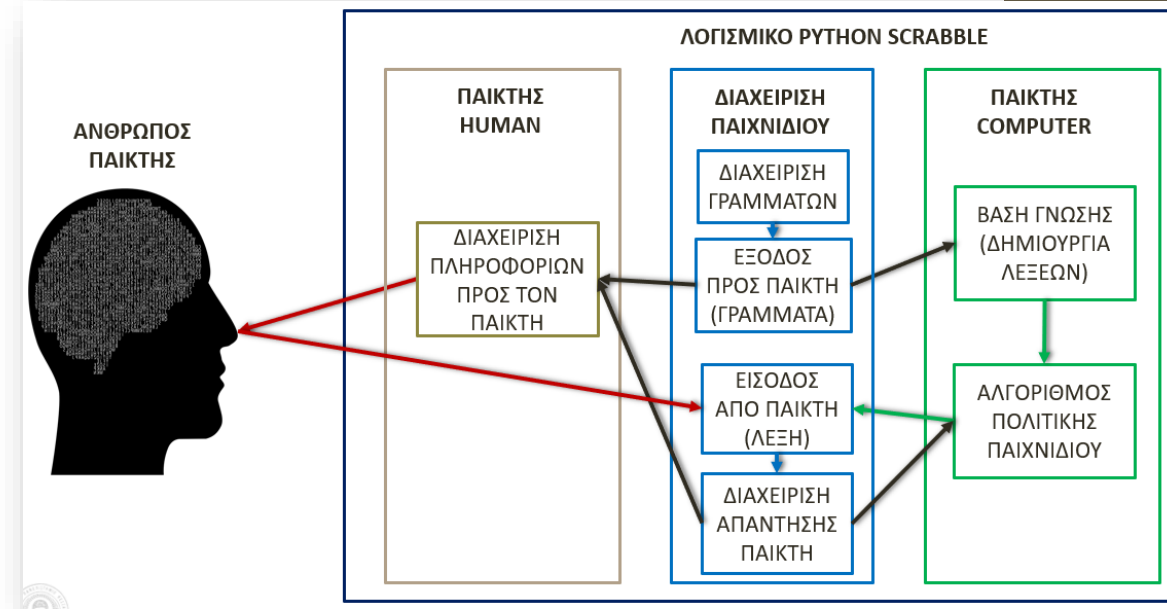
Αρχιτεκτονική 2/2

Human	Game	Computer
Ιδιότητες	Ιδιότητες	Ιδιότητες
<code>__init__</code> , play (extended)	<code>__init__</code> , setup, run, end	<code>__init__</code> , play (extended)



Εξηγήσεις για την Αρχιτεκτονική

- **Διαχείριση παιχνιδιού:** Κλάση που διαχειρίζεται τη συνολική λειτουργία του παιχνιδιού
 - Περιλαμβάνει μεθόδους για τη διαμοίραση γραμμάτων στους παίκτες, τη διαχείριση των απαντήσεών τους κλπ.
- **Παίκτης Computer:** Κλάση που διαχειρίζεται τις ενέργειες που πρέπει να μπορεί να κάνει ο παίκτης-computer
 - Περιλαμβάνει μεθόδους που να δημιουργούν λέξεις καθώς και μεθόδους πολιτικής, δηλ. λήψης αποφάσεων για τις ενέργειες του παίκτη στο παιχνίδι
- **Παίκτης Human:** Κλάση που διαχειρίζεται τις ενέργειες που πρέπει να μπορεί να κάνει το λογισμικό κατά την επικοινωνία του προς τον άνθρωπο-παίκτη
 - Π.χ. περιλαμβάνει μεθόδους που να παρουσιάζουν στη διεπαφή τις απαραίτητες πληροφορίες που χρειάζεται να γνωρίζει ο άνθρωπος-παίκτης στο παιχνίδι



(3) Κλάσεις



Οι 5 κλάσεις του παιχνιδιού

- Προγραμματιστικά η εφαρμογή θα πρέπει:
- **1) Να υλοποιεί τουλάχιστον τις παρακάτω 5 Κλάσεις:**
- **Game:** η κλάση που διαχειρίζεται το παιχνίδι συνολικά
- **SakClass:** η κλάση που διαχειρίζεται τα γράμματα («πλακίδια») του παιχνιδιού
- **Player:** βασική κλάση από την οποία παράγονται οι Human και Computer
- **Human:** κλάση παράγωγος της Player που περιλαμβάνει μεθόδους του παιχνιδιού για τον παίκτη Human
- **Computer:** κλάση παράγωγος της Player που περιλαμβάνει μεθόδους του παιχνιδιού για τον παίκτη Computer
- Πέρα από τις παραπάνω μπορείτε να υλοποιήσετε και άλλες κλάσεις αν θέλετε εσείς
- Περισσότερες πληροφορίες για τις κλάσεις δίνονται στις επόμενες διαφάνειες



Η κλάση SakClass – Βασικές μέθοδοι

sak

(αντικείμενο
τύπου SakClass)



getletters()

(βγάζει από το σακουλάκι για τον
παίκτη N γράμματα)

ΓΡΑΜΜΑΤΑ ΠΑΙΚΤΗ



putbackletters()

(επιστρέφει τα γράμματα του παίκτη
στο σακουλάκι)

randomize_sak()

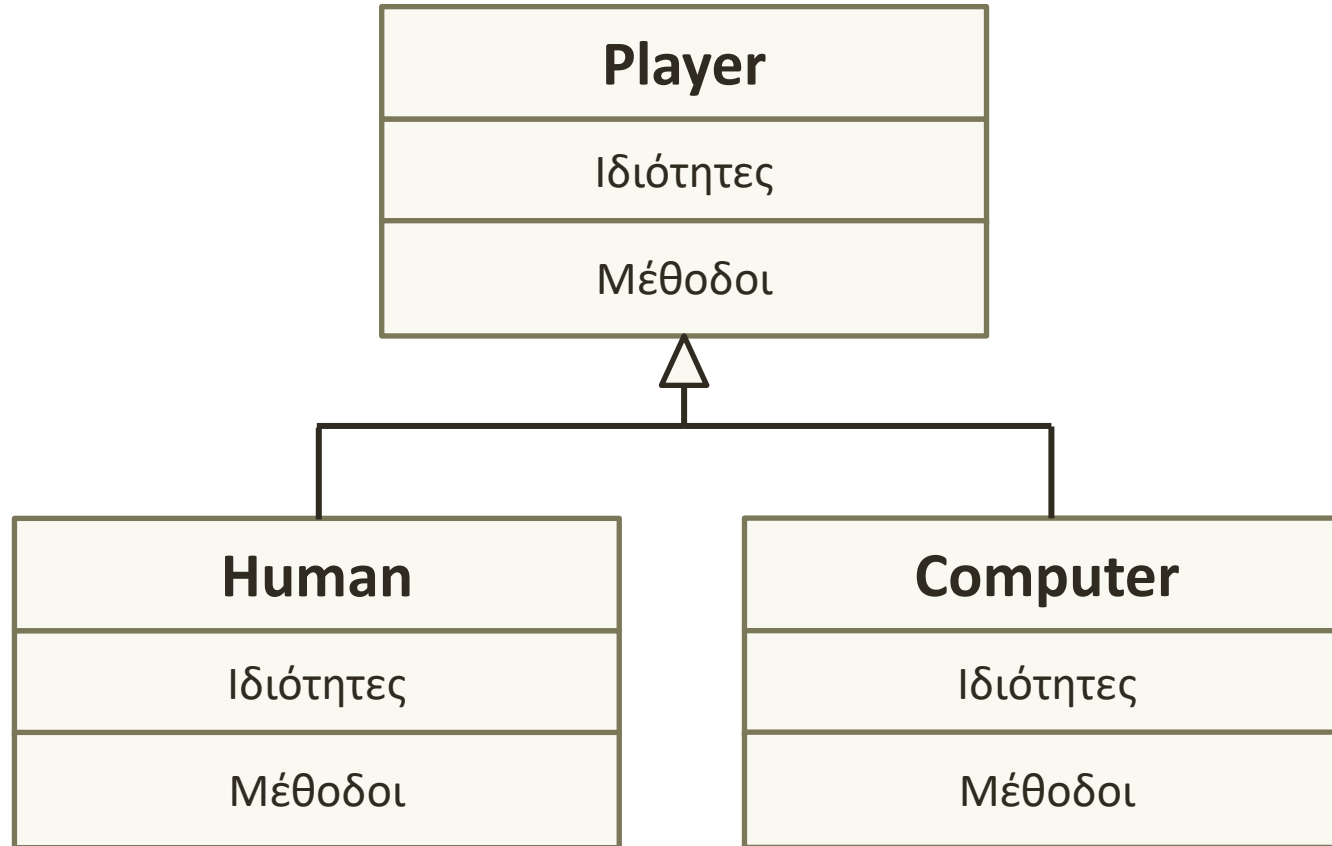
(«ετοιμάζει» δηλ.
«τυχαιοποιεί» το σακουλάκι
με τα γράμματα)

Η εικόνα παρουσιάζει μια πιθανή διαχείριση των γραμμάτων του παιχνιδιού από το αντικείμενο τύπου SakClass και τις μεθόδους που διαθέτει.

Η σχεδίαση αυτή είναι απλά ενδεικτική ώστε να γίνει κατανοητή η λογική της κλάσης SakClass.

Στον κώδικά σας μπορείτε να αναπτύξετε την κλάση με όποιες μεθόδους κρίνετε εσείς καταλληλότερες.

Κλάσεις: Player, Human & Computer



- Player: Βασική
- Human, Computer: Παράγωγες της Player

Κλάση: **Player**

Player
Ιδιότητες
<code>__init__</code> , <code>__repr__</code> <code>play</code>

- **Player**: Βασική
- Μέθοδοι: `init`, `repr`, `play` και όποιες άλλες κρίνετε εσείς
- Ιδιότητες: όποιες κρίνετε εσείς

Κλάση: **Human**

Human
Ιδιότητες
<code>__init__</code> , <code>__repr__</code> <code>play</code> (extended)

- **Human**: Παράγωγή της Player
- Μέθοδοι: `init`, `repr`, **`play`** (επεκτείνεται) και όποιες άλλες κρίνετε εσείς
- Ιδιότητες: όποιες κρίνετε εσείς

Κλάση: **Computer**

Computer
Ιδιότητες
<code>__init__</code> , <code>__repr__</code> <code>play</code> (extended)

- **Computer**: Παράγωγη της Player
- Μέθοδοι: `init`, `repr`, **play** (επεκτείνεται) και όποιες άλλες κρίνετε εσείς
- Η μέθοδος **play** θα υλοποιεί τον αλγόριθμο πολιτικής παιχνιδιού σύμφωνα με τον οποίο θα παίζει ο παίκτης-Computer

• Ιδιότητες: όποιες κρίνετε εσείς



Κλάση: Game

Game
Ιδιότητες
<code>__init__</code> , <code>__repr__</code> , <code>setup</code> , <code>run</code> , <code>end</code>

- **Game**: Βασική
- Μέθοδοι: `init`, `repr`, **`setup`**, **`run`**, **`end`** και όποιες άλλες κρίνετε εσείς
- Η μέθοδος **`setup`** κάνει τις απαραίτητες ενέργειες κατά το ξεκίνημα του παιχνιδιού
- Η μέθοδος **`run`** 'τρέχει' το παιχνίδι
- Η μέθοδος **`end`** κάνει τις απαραίτητες ενέργειες κατά το κλείσιμο του παιχνιδιού
- Ιδιότητες: όποιες κρίνετε εσείς

(4) Κανόνες του παιχνιδιού



Κανόνες του παιχνιδιού

1/3

- Η απλοποιημένη εκδοχή του Scrabble θα παιχτεί ως εξής:
- 1) Το παιχνίδι μπορεί να ξεκινά με μία εισαγωγική οθόνη που παρουσιάζει κατάλογο επιλογών στον παίκτη. Παράδειγμα μιας τέτοιας οθόνης βλέπετε δίπλα. Η ακριβής σχεδίαση είναι δική σας επιλογή.
- 2) Δημιουργείται ένα αρχικό «σακκουλάκι» που περιέχει συγκεκριμένο πλήθος γραμμάτων. Πληροφορίες για το πόσα είναι αυτά τα γράμματα υπάρχουν εδώ
- 3) Στην αρχή του παιχνιδιού δίνονται (αφαιρούνται από το σακκουλάκι) με τυχαίο τρόπο 7 γράμματα στον άνθρωπο παίκτη και 7 γράμματα στον παίκτη-Computer.
 - Τα γράμματα που θα έχει κάθε παίκτης θα είναι πάντοτε 7. Προς το τέλος του παιχνιδιού όταν δεν θα υπάρχουν άλλα γράμματα διαθέσιμα τότε μπορούν να είναι και λιγότερα από 7.

```
>>>
***** SCRABBLE *****
-----

1: Σκορ
2: Ρυθμίσεις
3: Παιχνίδι
q: Έξοδος
-----
```



- 4) Ο κάθε παίκτης στη σειρά του δημιουργεί μια λέξη που μπορεί να φτιάξει με τα γράμματα που διαθέτει και την δηλώνει στο παιχνίδι. Το λογισμικό εκτιμά αν η λέξη είναι αποδεκτή και αν ναι τότε:
 - Α) Πιστώνει στο σκορ του παίκτη τους αντίστοιχους πόντους της λέξης (που είναι το άθροισμα των πόντων που δίνει κάθε γράμμα)
 - Β) Συμπληρώνει τα γράμματα του παίκτη ώστε να έχει πάντοτε 7 (ή λιγότερα αν δεν υπάρχουν διαθέσιμα)
- 4) Αν ο παίκτης δεν βρίσκει αποδεκτή λέξη τότε το παιχνίδι του εμφανίζει ένα σχετικό μήνυμα και του δίνει επόμενη ευκαιρία.
- 5) Ο κάθε παίκτης στη σειρά του μπορεί να ζητήσει αλλαγή γραμμάτων (αλλάζουν και τα 7 ή όσα γράμματα έχει διαθέσιμα) και χάνει τη σειρά του.



- 6) Το παιχνίδι συνεχίζεται με παρόμοιες κινήσεις από όσους παίκτες παίζουν μέχρις ότου να τελειώσουν τα γράμματα στο σακκουλάκι και κανείς να μην μπορεί να σχηματίσει αποδεκτή λέξη ή να παραιτηθεί κάποιος παίκτης από το παιχνίδι.
- 7) Τότε το παιχνίδι ανακοινώνει τη λήξη του παιχνιδιού και τον νικητή και παρουσιάζει το τελικό σκορ.
- 8) Ένας παίκτης ζητά αλλαγή γραμμάτων πατώντας το πλήκτρο 'P' (επάνω δεξιά στο αγγλικό πληκτρολόγιο, δηλ. το ελληνικό 'Π')
- 9) Ένας παίκτης παραιτείται πατώντας το πλήκτρο 'Q'



Περιβάλλον ανάπτυξης

- Η εφαρμογή σε κώδικα Python θα πρέπει να μπορεί να εκτελείται είτε σε **περιβάλλον IDLE** είτε σε **περιβάλλον Jupyter Notebook** και να εφαρμόζει τις αρχές αντικειμενοστρεφούς προγραμματισμού (Α.Π.) που διδάχθηκαν στο μάθημα.
- Η εφαρμογή θα χρησιμοποιεί μόνον τις ενσωματωμένες βιβλιοθήκες της στάνταρ Python (δηλ. δεν χρειάζεται να εγκαταστήσετε ή να χρησιμοποιήσετε πρόσθετες βιβλιοθήκες).
- Όλες οι πληροφορίες κατά την εξέλιξη του παιχνιδιού θα εμφανίζονται με μηνύματα χαρακτήρων (character mode), δηλ. η εφαρμογή δεν θα χρησιμοποιεί γραφικά.
- Πχ. μια πιθανή εμφάνιση μηνυμάτων από το παιχνίδι και πληκτρολόγησης λέξης από τον παίκτη, μπορεί να είναι όπως παρακάτω:

```
*****
```

```
*** Παίκτης: Stavros *** Σκορ: 0
```

```
>>> Γράμματα: ['A', 'N', 'Ψ', 'Η', 'Ρ', 'Υ', 'Τ']
```

```
ΛΕΞΗ:
```

Παράδειγμα μηνυμάτων εξέλιξης παιχνιδιού

- Εδώ βλέπετε και άλλα παραδείγματα για το πώς μπορεί να εξελίσσεται το παιχνίδι στην οθόνη
- Δεν είστε υποχρεωμένοι να ακολουθήσετε αυτό το παράδειγμα
- Μπορείτε να διαμορφώσετε τα μηνύματα όπως θέλετε εσείς αρκεί να είναι κατανοητά από τους παίκτες

```
*****
*** Παίκτης: Stavros *** Σκορ: 0
>>> Γράμματα: ['Α', 'Ν', 'Ψ', 'Η', 'Ρ', 'Υ', 'Τ']

ΛΕΞΗ: ΑΥΤΗ
Πόντοι Λέξης: 5
*** Παίκτης: Stavros *** Σκορ: 5
>>> Γράμματα: ['Ν', 'Ψ', 'Ρ', 'Β', 'Ν', 'Η', 'Ι']

*****
*****
*** Παίκτης: PC *** Σκορ: 0
>>> Γράμματα: ['Ρ', 'Ε', 'Α', 'Ε', 'Τ', 'Α', 'Λ']

Παίζω τη Λέξη: ΑΕΡΑΤΕ
Πόντοι Λέξης: 7
*** Παίκτης: PC *** Σκορ: 7
>>> Γράμματα: ['Λ', 'Ο', 'Ρ', 'Ο', 'Σ', 'Θ', 'Ι']

*****
*****
*** Παίκτης: Stavros *** Σκορ: 5
>>> Γράμματα: ['Ν', 'Ψ', 'Ρ', 'Β', 'Ν', 'Η', 'Ι']

ΛΕΞΗ: 
```

Λέξη που
πληκτρολόγησε ο
άνθρωπος παίκτης

Νέα ενημερωμένη
ομάδα γραμμάτων
του ανθρώπου παίκτη

Λέξη που έπαιξε ο
παίκτης-Computer

Νέα ενημερωμένη
ομάδα γραμμάτων
του παίκτη-Computer

(5) Θέματα ανάπτυξης κώδικα



5.1 Δομή δεδομένων για τις λέξεις



Δομή δεδομένων για τις λέξεις: Λεξικό (dictionary), Λίστα (list) ή Σύνολο (set);

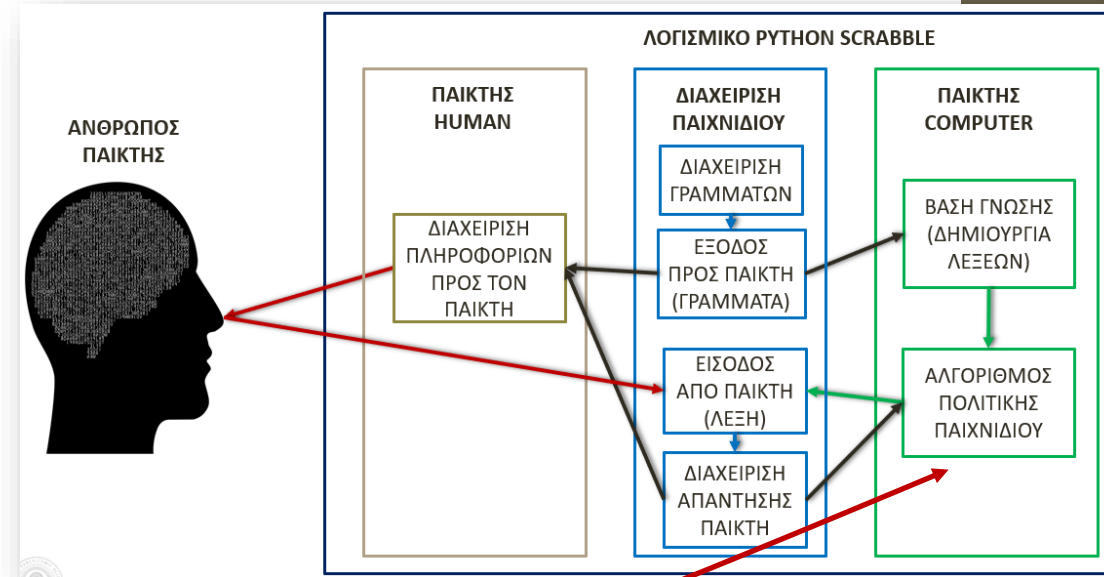
- Το πρόγραμμά σας στο ξεκίνημα θα πρέπει να “φορτώνει” τις λέξεις της ελληνικής γλώσσας από το αρχείο **greek7.txt** (διαθέσιμο στη σελίδα του μαθήματος) και να τις μεταφέρει σε μια κατάλληλη δομή στον κώδικα ώστε να μπορούν να γίνουν οι απαραίτητοι έλεγχοι στο παιχνίδι.
- Θα πρέπει να αποφασίσετε για τη **δομή δεδομένων** που θα περιέχει τις λέξεις της γλώσσας που θα συμβουλεύεται το πρόγραμμά σας.
- Από τη είδος της δομής θα εξαρτηθεί η **αποδοτικότητα του κώδικά** σας. Ο κώδικάς σας θα πρέπει να χρησιμοποιεί δομή η οποία επιτυγχάνει χρονική πολυπλοκότητα τάξης $O(1)$ (και όχι $O(n)$ γιατί αυτό καθυστερεί το παιχνίδι).
- Προσέξτε το σημείο αυτό, δίνουμε την απάντηση στο εργαστήριο.

5.2 Αλγόριθμος πολιτικής παιχνιδιού



Επιλέξτε τον αλγόριθμο με τον οποίο θα παίζει ο Η/Υ (κλάση Computer)

- **Αλγόριθμος πολιτικής παιχνιδιού** είναι ο αλγόριθμος σύμφωνα με τον οποίο παίρνει αποφάσεις η μηχανή (Η/Υ) ώστε να κάνει κινήσεις στη διάρκεια του παιχνιδιού.
- Από τον αλγόριθμο αυτό εξαρτάται το επίπεδο δυσκολίας και η γενική ποιότητα του παιχνιδιού της μηχανής.
- Μπορείτε:
- Α) Να επιλέξετε έναν από τους αλγορίθμους που προτείνονται στη συνέχεια
- Β) Να καθορίσετε έναν δικό σας
 - (στην περίπτωση αυτή ενημερώστε με έγκαιρα ώστε να εξετάσω τον αλγόριθμο που προτείνετε και να σας δώσω έγκριση)



- Πρόκειται για ομάδα τριών αλγορίθμων που πρέπει όλοι να υλοποιηθούν (αν επιλέξετε αυτή την περίπτωση) και να προσφέρονται ως επιλογή στον άνθρωπο παίκτη για να ρυθμίσει το επίπεδο του Η/Υ στο ξεκίνημα του παιχνιδιού.
- Α) **MIN Letters**: Το πρόγραμμα δημιουργεί όλες τις δυνατές μεταθέσεις (permutations) των γραμμάτων που διαθέτει ο Η/Υ ξεκινώντας από 2 και ανεβαίνοντας μέχρι τα 7 γράμματα. Για κάθε μετάθεση ελέγχει αν είναι αποδεκτή λέξη και παίζει την πρώτη αποδεκτή λέξη που θα εντοπίσει.
- Β) **MAX Letters**: Όπως και στο Α αλλά το πρόγραμμα ξεκινά από τις μεταθέσεις των γραμμάτων ανά 7 και κατεβαίνει προς το 2. Παίζει πάλι την πρώτη αποδεκτή λέξη αλλά τώρα αυτή με τα περισσότερα γράμματα.
- Γ) **SMART**: Όπως και στο Α αλλά εξαντλεί όλες τις μεταθέσεις 2 ως και 7 γραμμάτων χωρίς να σταματά. Βρίσκει τις αποδεκτές λέξεις και στο τέλος παίζει τη λέξη που δίνει τους περισσότερους βαθμούς.
- Οι μεταθέσεις μιας λίστας αντικειμένων μπορούν εύκολα να υπολογιστούν μέσω της συνάρτησης itertools.permutations()

- Ο αλγόριθμος παιχνιδιού SMART-FAIL έχει την εξής μορφή:
- **SMART:** Καλείται πρώτα ο αλγόριθμος SMART (όπως εξηγήθηκε στο σενάριο 1) και δημιουργεί μια λίστα με πιθανές λέξεις που μπορούν να παιχτούν.
- **FAIL:** Στη συνέχεια καλείται ο FAIL. Όπως ένα άνθρωπος δεν βρίσκει πάντοτε τη βέλτιστη λέξη ο αλγόριθμος FAIL εισάγει ένα βαθμό αποτυχίας στο να παίξει τη βέλτιστη λέξη που έχει βρει ο SMART.
- Ο αλγόριθμος FAIL παίρνει ως είσοδο τη λίστα πιθανών λέξεων που παράγει ο SMART και επιλέγει να παίξει πχ. τη 2η καλύτερη ή την 3η καλύτερη λέξη στη λίστα (αντί της βέλτιστης πρώτης) ανάλογα πώς θα τον γράψετε.
- Μπορείτε να αποφασίσετε εσείς τις λεπτομέρειες του αλγορίθμου FAIL έτσι ώστε ο FAIL να προσομοιώνει τη μνήμη/ικανότητα ενός ανθρώπου που δεν γνωρίζει όλες τις λέξεις ή δεν μπορεί να βρει πάντοτε την καλύτερη δυνατή λέξη.

- Ο αλγόριθμος παιχνιδιού SMART-LEARN έχει την εξής μορφή:
- **SMART:** Ο αλγόριθμος όπως εξηγήθηκε στο σενάριο 1
- **LEARN:** Ο υπολογιστής «μαθαίνει» νέες λέξεις, δηλ. ο αλγόριθμος προσομοιώνει έναν παίκτη ο οποίος μαθαίνει καθώς παίζει.
- Ο αλγόριθμος «μαθαίνει» από τον άνθρωπο-παίκτη. Δηλ. αν ο άνθρωπος παίκτης σε κάποια κίνηση εισάγει μια λέξη που δεν υπάρχει στο αρχείο greek7.txt (και είναι μέχρι 7 γράμματα) τότε αντί να απορριφθεί ερωτάται ο παίκτης αν θέλει να συμπεριληφθεί στο αρχείο λέξεων για τη συνέχεια.
- Εφόσον ο παίκτης επιλέξει 'ΝΑΙ' η λέξη συμπεριλαμβάνεται στη δομή λέξεων του τρέχοντος παιχνιδιού ώστε να αναγνωρίζεται στη συνέχεια και από τον παίκτη – computer.
- Στο σενάριο αυτό θα πρέπει στο τέλος του παιχνιδιού να ενημερώσετε το αρχείο greek7.txt με τις νέες λέξεις ώστε να είναι διαθέσιμες σε επόμενη παρτίδα.
- Μπορείτε να σχεδιάσετε εσείς τις τεχνικές-προγραμματιστικές λεπτομέρειες του σεναρίου

ΑΛΓΟΡΙΘΜΟΣ 4

Smart-Teach

- Ο αλγόριθμος παιχνιδιού SMART-TEACH έχει την εξής μορφή:
- **SMART:** Ο αλγόριθμος όπως εξηγήθηκε στο σενάριο 1
- **TEACH:** Ο υπολογιστής «διδάσκει» τον παίκτη, δηλ. τον ενημερώνει ποια θα ήταν η καλύτερη λέξη να παίξει.
- Όταν είναι σειρά του παίκτη-ανθρώπου να παίξει, ο υπολογιστής εκτελεί επίσης τον αλγόριθμο SMART.
- Αφού παίξει ο παίκτη κάποια λέξη το παιχνίδι ενημερώνει τον παίκτη αν η λέξη που έπαιξε είναι η καλύτερη δυνατή. Αν δεν είναι, τότε τον ενημερώνει ποια θα ήταν η καλύτερη ή και η 2η καλύτερη λέξη που θα μπορούσε να παίξει.
- Μπορείτε να σχεδιάσετε εσείς τις τεχνικές-προγραμματιστικές λεπτομέρειες του σεναρίου.

Αλγόριθμοι και Ρυθμίσεις Παιχνιδιού

- Κατά το ξεκίνημα του παιχνιδιού θα πρέπει να μπορεί ο άνθρωπος παίκτης να επιλέξει με ποιόν αλγόριθμο θα παίξει ο υπολογιστής, εφόσον φυσικά ο αλγόριθμος πολιτικής που επιλέξατε για τον Η/Υ δίνει αυτή τη δυνατότητα.
- Πχ. αν εφαρμόσατε τον Min-Max-Smart τότε από τις ρυθμίσεις στην αρχική οθόνη του παιχνιδιού θα πρέπει ο παίκτης να μπορεί να καθορίσει αν ο Η/Υ θα παίξει με την επιλογή Min ή Max ή Smart

```
>>>  
***** SCRABBLE *****  
-----  
1: Σκορ  
2: Ρυθμίσεις  
3: Παιχνίδι  
q: Έξοδος  
-----
```

5.3 Βιβλιοθήκη & import



Module (Βιβλιοθήκη, άρθρωμα)

- Ένα **'module'** (**'βιβλιοθήκη'** ή **'άρθρωμα'**) είναι ένα αρχείο .py στο οποίο έχετε αποθηκεύσει κώδικα που κάνει συγκεκριμένες εργασίες, πχ. κλάσεις ή και απλές συναρτήσεις
- Μια τέτοια βιβλιοθήκη συνδέεται με το κύριο πρόγραμμά σας με εντολή **import**
- Πχ. ας υποθέσουμε πως έχετε γράψει την κλάση SakClass και την έχετε αποθηκεύσει σε ένα αρχείο classes.py
- Τότε στο κύριο πρόγραμμά σας μπορείτε να γράψετε:

import classes

- Και να δημιουργήσετε το αντικείμενο sak τύπου SakClass γράφοντας:

sak = classes.SakClass()

5.4 Άλλα θέματα ανάπτυξης κώδικα



Διαβάστε για τα παρακάτω θέματα ανάπτυξης κώδικα στο αρχείο:

07-Python-ScriptsForScrabble-2026.pdf

- Τα θέματα αυτά τα αναπτύσσουμε στο εργαστήριο του μαθήματος
- 1) Αρχείο λέξεων της Ελληνικής γλώσσας
 - Πώς να χρησιμοποιήσετε το αρχείο greek7.txt
- 2) Δομή δεδομένων για τα γράμματα του παιχνιδιού
 - Ποιά δομή να επιλέξετε με ώστε να έχετε χρονική πολυπλοκότητα $O(1)$
- 3) Ενδιάμεση αποθήκευση: pickle vs json
 - Πώς να κάνετε ενδιάμεση αποθήκευση δεδομένων με pickle ή json βιβλιοθήκη
- 4) itertools: βιβλιοθήκη για συνδυαστική ανάλυση
 - Πώς να χρησιμοποιήσετε αλγόριθμους συνδυαστικής ανάλυσης της βιβλιοθήκης itertools
- 5) random: βιβλιοθήκη για διαχείριση ψευδοτυχαίων αριθμών
 - Πώς να διαχειριστείτε ψευδοτυχαίους αριθμούς
- 6) get: ασφαλής πρόσβαση σε δεδομένα dictionary
 - Πώς να κάνετε 'ασφαλή' πρόσβαση στα δεδομένα dictionary (δηλ. να μην σταματά η εκτέλεση του κώδικα αν κάποιο κλειδί που ψάχνετε δεν περιλαμβάνεται στο dictionary)
- 7) Επικοινωνία κλάσεων μεταξύ τους
 - Πώς να εφαρμόσετε την επικοινωνία των κλάσεων κατά την εκτέλεση του παιχνιδιού

Πώς θα αποθηκεύσετε τον κώδικά σας

- Α) Ο κώδικας των κλάσεων θα βρίσκεται σε **ένα ξεχωριστό αρχείο** το οποίο θα ονομάσετε **classes.py**
- Β) Ο κώδικας του κύριου προγράμματος θα βρίσκεται σε ένα αρχείο με όνομα **main-AEM.py**
 - Όπου ΑΕΜ θα γράψετε τον αριθμό του ΑΕΜ σας, πχ. main-1234.py

(6) Αξιολόγηση εργασίας



6.1 Τεκμηρίωση & docstring



Τεκμηρίωση (με μορφή docstring)

- Συμπεριλάβετε τεκμηρίωση του κώδικά σας σε μορφή **docstring** ενσωματωμένο σε μια απλή συνάρτηση (όχι μέθοδο κλάσης) με όνομα '**documentation**'
- Αποθηκεύστε τη συνάρτηση `documentation` μέσα στο αρχείο **classes.py** (κατά προτίμηση στην αρχή ώστε να την εντοπίσω εύκολα).
- Αν δεν έχετε χρησιμοποιήσει ποτέ docstrings δείτε την επόμενη διαφάνεια για βασικές πληροφορίες και σύνδεσμο προς σχετικό υλικό.
- Αν γνωρίζετε τι είναι το docstring πηγαίνετε στην επόμενη υποενότητα «6.2 Κριτήρια Αξιολόγησης» και δείτε τι πληροφορίες πρέπει να συμπεριλάβετε στο docstring τεκμηρίωσης.

Docstring

- Το docstring (“documentation string”) είναι ο απλούστερος τρόπος για να συμπεριλάβουμε δομημένα σχόλια τεκμηρίωσης στον κώδικα Python.
- Γράφεται αμέσως μετά την «υπογραφή» της συνάρτησης (ή κλάσης κλπ.) και ανοίγει-κλείνει με **τριπλά εισαγωγικά** (όπως βλέπετε στο παράδειγμα της εικόνας).
- Συνήθως η τεκμηρίωση περιλαμβάνει:
 1. Σύντομη περιγραφή της συνάρτησης/κλάσης
 2. Περιγραφή των παραμέτρων εισόδου (αν υπάρχουν)
 3. Περιγραφή των τιμών που επιστρέφονται (αν υπάρχουν)
 4. Πληροφορίες για πιθανές εξαιρέσεις (exceptions) που μπορεί να προκαλέσει η συνάρτηση

```
def function_name(parameters):  
    """  
    Function description  
  
    Parameters:  
    - parameter1: description of the parameter  
    - parameter2: description of the parameter  
  
    Returns:  
    - return_type: description of the return type  
    """  
    # Function body
```

Παράδειγμα από:

<https://www.luisllamas.es/en/python-documenting-code-docstrings/>

6.2 Κριτήρια Αξιολόγησης



Πριν υποβάλετε την εργασία σας ελέγξτε πρώτα ότι γράψατε στο docstring της συνάρτησης documentation τις εξής πληροφορίες:

1. Ποιες **κλάσεις** έχετε υλοποιήσει στον κώδικά σας
2. Ποιά **κληρονομικότητα** έχετε υλοποιήσει (Π.χ. Ποιά η βασική κλάση και ποιές οι παράγωγες που έχετε υλοποιήσει)
3. Ποιά **επέκταση μεθόδων** έχετε υλοποιήσει (π.χ. Ποιές μέθοδοι επεκτείνουν την ανώτερη στις κλάσεις που δημιουργήσατε) και τι είδους **πολυμορφισμό** δημιουργείτε έτσι
4. Σε ποιά **δομή** (λεξικό-dictionary, λίστα-list ή σύνολο-set) οργανώνει η εφαρμογή σας τις λέξεις της γλώσσας στη διάρκεια του παιχνιδιού
5. Ποιόν **αλγόριθμο πολιτικής** παιχνιδιού υλοποιήσατε για να παίξει ο παίκτης-Computer
6. Πώς εφαρμόσατε το **πρότυπο/μοτίβο λογισμικού “Mediator”** για την επικοινωνία των κλάσεων μεταξύ τους

Προαιρετικό: Αν έχετε εφαρμόσει υπερφόρτωση τελεστών και decorators και σε ποιο σημείο του κώδικα (θα ληφθεί θετικά στην αξιολόγηση)

documentation()

- Η συνάρτηση documentation() στον κώδικά σας θα μοιάζει όπως φαίνεται στην εικόνα.
- Στη θέση των αποσιωπητικών (...) εσείς θα συμπληρώσετε τις σχετικές πληροφορίες για τον κώδικά σας.

```
def documentation():  
    """  
    - Κλάσεις:...  
    - Κληρονομικότητα: ...  
    - Επέκταση μεθόδων: ...  
    - Υπερφόρτωση τελεστών:...  
    - Decorators: ...  
    - Δομή λέξεων:...  
    - Αλγόριθμος πολιτικής παιχνιδιού:...  
    - Πρότυπο Mediator:...  
    """  
  
    pass
```

- Η αξιολόγηση της εργασίας θα γίνει με βάση τα παρακάτω κριτήρια

α/α	ΚΡΙΤΗΡΙΟ	ΣΧΟΛΙΑ - ΕΞΗΓΗΣΕΙΣ
1	ΚΛΑΣΕΙΣ	- Αν έχουν υλοποιηθεί οι 5 τουλάχιστον κλάσεις που προβλέπει η εργασία (Game, SakClass, Player, Human, Computer)
2	ΚΛΗΡΟΝΟΜΙΚΟΤΗΤΑ	- Αν έχει υλοποιηθεί κληρονομικότητα κλάσεων. Αν οι κλάσεις Game, Human & Computer συνεργάζονται σωστά ώστε να παιχτεί το παιχνίδι.
3	ΕΠΕΚΤΑΣΗ ΜΕΘΟΔΟΥ & ΠΟΛΥΜΟΡΦΙΣΜΟΣ	- Αν έχει υλοποιηθεί επέκταση μεθόδου και πολυμορφισμός τουλάχιστον σε μία περίπτωση. <i>Η υπερφόρτωση τελεστών και η χρήση decorators δεν είναι υποχρεωτική αλλά αν υλοποιηθεί θα ληφθεί υπόψη θετικά στη συνολική βαθμολογία.</i>
4	ΔΟΜΗ ΛΕΞΕΩΝ	- Αν έχει χρησιμοποιηθεί δομή με χρονική πολυπλοκότητα $O(1)$ για την αποθήκευση και αναζήτηση λέξεων στη διάρκεια του παιχνιδιού.

- Συνέχεια κριτηρίων αξιολόγησης ... →

α/α	ΚΡΙΤΗΡΙΟ	ΣΧΟΛΙΑ - ΕΞΗΓΗΣΕΙΣ
5	ΑΛΓΟΡΙΘΜΟΣ COMPUTER	- Αν έχει υλοποιηθεί σωστά ένας αλγόριθμος πολιτικής παιχνιδιού για τον παίκτη-Computer
6	ΤΕΚΜΗΡΙΩΣΗ	- Αν έχουν γραφεί σωστά όλες οι πληροφορίες που ζητά η τεκμηρίωση στο docstring της συνάρτησης documentation()
7	ΠΡΟΤΥΠΟ Mediator	- Αν έχει εφαρμοστεί σωστά το πρότυπο Mediator για την επικοινωνία των κλάσεων του παιχνιδιού μεταξύ τους
8	ΕΚΤΕΛΕΣΗ ΚΩΔΙΚΑ ΓΕΝΙΚΑ	- Αν το παιχνίδι παίζεται –σε γενικές γραμμές- μέσα στο πλαίσιο των οδηγιών που δόθηκαν χωρίς σφάλματα (bugs) και χτυπητές ελλείψεις. - Σε περίπτωση που υπάρχουν σφάλματα που μπορεί να διορθωθούν μπορεί να σας καλέσω να διορθώσετε την εργασία και να επανυποβάλετε διορθωμένη έκδοση.

- Μια πλήρης εργασία βαθμολογείται με **40/100** από τις συνολικές μονάδες του μαθήματος.
- Αποτυχία σε κάποιο ή κάποια κριτήρια μπορεί να σημαίνει μείωση της βαθμολογίας της εργασίας



(7) Παραδοτέο



Μορφή Παραδοτέου

- Παραδοτέο της εργασίας σας είναι **1 συμπιεσμένο αρχείο (zip ή rar)** που περιλαμβάνει:
 - (α) Το αρχείο **classes.py** που περιλαμβάνει τις κλάσεις της εργασίας σας και τη συνάρτηση `documentation()`
 - (β) Το αρχείο **main-AEM.py** που περιλαμβάνει το κύριο πρόγραμμα
 - (γ) Κάθε άλλο απαραίτητο αρχείο όπως εσείς θεωρείτε σωστό για την εκτέλεση του κώδικα (π.χ. αν χρειάζεται κάποιο αρχείο τύπου json) (προσοχή: **OXI to greek7.txt**)
- >>> Τα παραπάνω αρχεία τα συμπιέζετε σε αρχείο zip (κατά προτίμηση) ή rar
- >>> Ονομάζετε το συμπιεσμένο αρχείο σύμφωνα με το σχήμα: <ΕΠΩΝΥΜΟ-AEM>.zip (π.χ. ΔΗΜΗΤΡΙΑΔΗΣ-1234.zip)
- Υποβάλετε μέσω elearning μέχρι την προθεσμία που θα ανακοινωθεί στην εξεταστική που θέλετε να παραδώσετε την εργασία

Πηγές στο διαδίκτυο

- [Σκράμπλ – Βικιπαίδεια](#): Τι είναι το Scrabble και γενικοί κανόνες του παιχνιδιού
- Python **itertools** (ενσωματωμένη βιβλιοθήκη) – ειδικά τη συνάρτηση [itertools.permutations\(\)](#)
- [Built in types](#) (ειδικά τις μεθόδους συμβολοσειρών: [strip](#), [split](#))
- Βιβλιοθήκη [random](#) για διαχείριση ψευδοτυχαίων αριθμών